

Statistical Analysis and Predictive Modeling using R

Pima Indians Diabetes

Suruthi D

28 October 2026

Table of Contents

1. Setup and packages 3
2. Dataset identification 3
3. Exploratory statistical analysis 4
3.1 Summary statistics and missing values 4
3.2 Target balance 6
3.3 Distributions 7
3.4 Normality checks 9
3.5 Correlation analysis 10
3.6 Hypothesis tests 11
4. Model building 13
4.1 Train / test split 13
4.2 Cross-validation setup 14
4.3 Model 1: Logistic regression 14
4.4 Model 2: Random forest (comparison model) 15
4.5 Cross-validated comparison 16
5. Evaluation on the held-out test set 17
5.1 Confusion matrix plot 19
5.2 ROC curves and AUC 21
6. Diagnostic analysis 21
6.1 Logistic regression: odds ratios 21
6.2 Multicollinearity (VIF, computed manually) 22
6.3 Residual diagnostics 22
6.4 Variable importance 25
7. Optimization and improvement 27
7.1 Threshold tuning (improving sensitivity) 27
7.2 Other ideas to try 27
8. Interpretation of Results 28
9. Key Insights & Key Features 28
10. Conclusion & Limitation 29
11.Screenshots of the RStudio Workflow 30
Appendix 31

1. Setup and packages

```
pkgs <- c("mlbench", "ggplot2", "dplyr", "tidyr", "caret",
          "pROC", "corrplot", "randomForest")
for (p in pkgs) {
  if (!requireNamespace(p, quietly = TRUE)) {
    install.packages(p, repos = "https://cloud.r-project.org")
  }
}
invisible(lapply(pkgs, library, character.only = TRUE))
set.seed(123)
```

2. Dataset identification

Dataset:

`PimaIndiansDiabetes2` from the `mlbench` package (originally from the National Institute of Diabetes and Digestive and Kidney Diseases). It holds medical measurements for 768 women of Pima Indian heritage aged 21 or older. The target is `diabetes` (pos / neg).

Why this dataset:

This dataset was chosen because it has a clear binary outcome (diabetes yes/no) with eight clinical predictors, which suits both hypothesis testing and classification. It is a well-known public benchmark, small enough for 10-fold cross-validation, and it contains missing values (insulin 48.7%, triceps 29.6%), which makes the preprocessing decisions realistic.

```
df <- NULL

# Try 1: from the mlbench package
df <- tryCatch(mlbench::PimaIndiansDiabetes2, error = function(e) NULL)

# Try 2: download the raw data if the package version fails
if (is.null(df)) {
  df <- tryCatch({
    url <- "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-
indians-diabetes.data.csv"
    d <- read.csv(url, header = FALSE)
    names(d) <- c("pregnant", "glucose", "pressure", "triceps",
                  "insulin", "mass", "pedigree", "age", "diabetes")
    d$diabetes <- factor(ifelse(d$diabetes == 1, "pos", "neg"),
                         levels = c("neg", "pos"))
    cols <- c("glucose", "pressure", "triceps", "insulin", "mass")
    d[cols] <- lapply(d[cols], function(x) ifelse(x == 0, NA, x))
    d
  }, error = function(e) NULL)
```

```
}

if (is.null(df)) stop("Dataset could not be loaded. Install mlbench and resta
rt R.")

df <- as.data.frame(df)
dim(df)

## [1] 768   9

str(df)

## 'data.frame':    768 obs. of  9 variables:
##  $ pregnant: int  6 1 8 1 0 5 3 10 2 8 ...
##  $ glucose : int  148 85 183 89 137 116 78 115 197 125 ...
##  $ pressure: int  72 66 64 66 40 74 50 NA 70 96 ...
##  $ triceps : int  35 29 NA 23 35 NA 32 NA 45 NA ...
##  $ insulin : int  NA NA NA 94 168 NA 88 NA 543 NA ...
##  $ mass    : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 NA ...
##  $ pedigree: num  0.627 0.351 0.672 0.167 2.288 ...
##  $ age     : int  50 31 32 21 33 30 26 29 53 54 ...
##  $ diabetes: Factor w/ 2 levels "neg","pos": 2 1 2 1 2 1 2 1 2 2 ...

head(df)

##   pregnant glucose pressure triceps insulin mass pedigree age diabetes
## 1        6     148       72      35      NA 33.6    0.627  50      pos
## 2        1      85       66      29      NA 26.6    0.351  31      neg
## 3        8     183       64      NA      NA 23.3    0.672  32      pos
## 4        1      89       66      23      94 28.1    0.167  21      neg
## 5        0     137       40      35     168 43.1    2.288  33      pos
## 6        5     116       74      NA      NA 25.6    0.201  30      neg
```

3. Exploratory statistical analysis

3.1 Summary statistics and missing values

```
summary(df)

##     pregnant         glucose         pressure         triceps
##  Min.   : 0.000   Min.   : 44.0   Min.   : 24.00   Min.   : 7.00
##  1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 64.00   1st Qu.:22.00
##  Median : 3.000   Median :117.0   Median : 72.00   Median :29.00
##  Mean   : 3.845   Mean   :121.7   Mean   : 72.41   Mean   :29.15
##  3rd Qu.: 6.000   3rd Qu.:141.0   3rd Qu.: 80.00   3rd Qu.:36.00
##  Max.   :17.000   Max.   :199.0   Max.   :122.00   Max.   :99.00
##                   NAs    :5       NAs    :35       NAs    :227
##     insulin           mass          pedigree           age        diabete
s
##  Min.   : 14.00   Min.   :18.20   Min.   :0.0780   Min.   :21.00   neg:500
```

```
##  1st Qu.: 76.25   1st Qu.:27.50   1st Qu.:0.2437   1st Qu.:24.00   pos:268

##  Median :125.00   Median :32.30   Median :0.3725   Median :29.00

##  Mean   :155.55   Mean   :32.46   Mean   :0.4719   Mean   :33.24

##  3rd Qu.:190.00   3rd Qu.:36.60   3rd Qu.:0.6262   3rd Qu.:41.00

##  Max.   :846.00   Max.   :67.10   Max.   :2.4200   Max.   :81.00

##  NAs    :374      NAs    :11
```

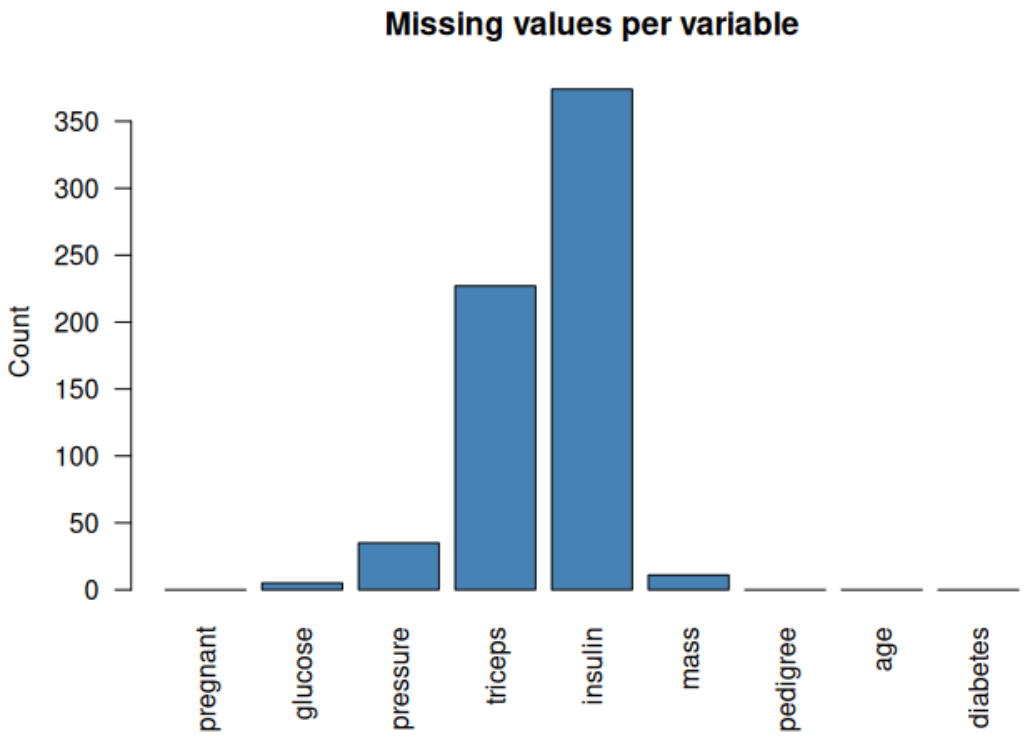
```
colSums(is.na(df))
```

```
## pregnant  glucose pressure  triceps  insulin     mass pedigree      age
##        0        5       35      227      374       11        0        0
## diabetes
##        0
```

```
round(colMeans(is.na(df)) * 100, 1)   # % missing per column
```

```
## pregnant  glucose pressure  triceps  insulin     mass pedigree      age
##      0.0      0.7      4.6     29.6     48.7      1.4      0.0      0.0
## diabetes
##      0.0
```

```
miss <- colSums(is.na(df))
barplot(miss, las = 2, col = "steelblue",
        main = "Missing values per variable", ylab = "Count")
```



3.2 Target balance

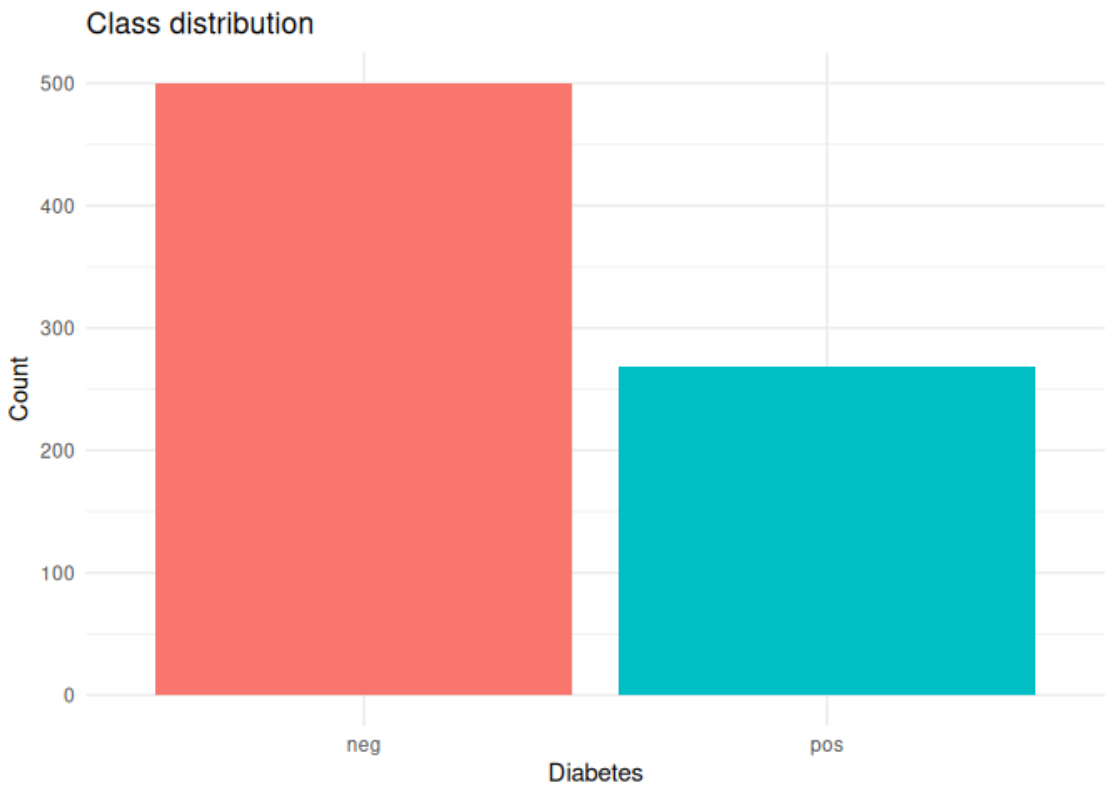
```
table(df$diabetes)

##
## neg pos
## 500 268

round(prop.table(table(df$diabetes)), 3)

##
##   neg   pos
## 0.651 0.349

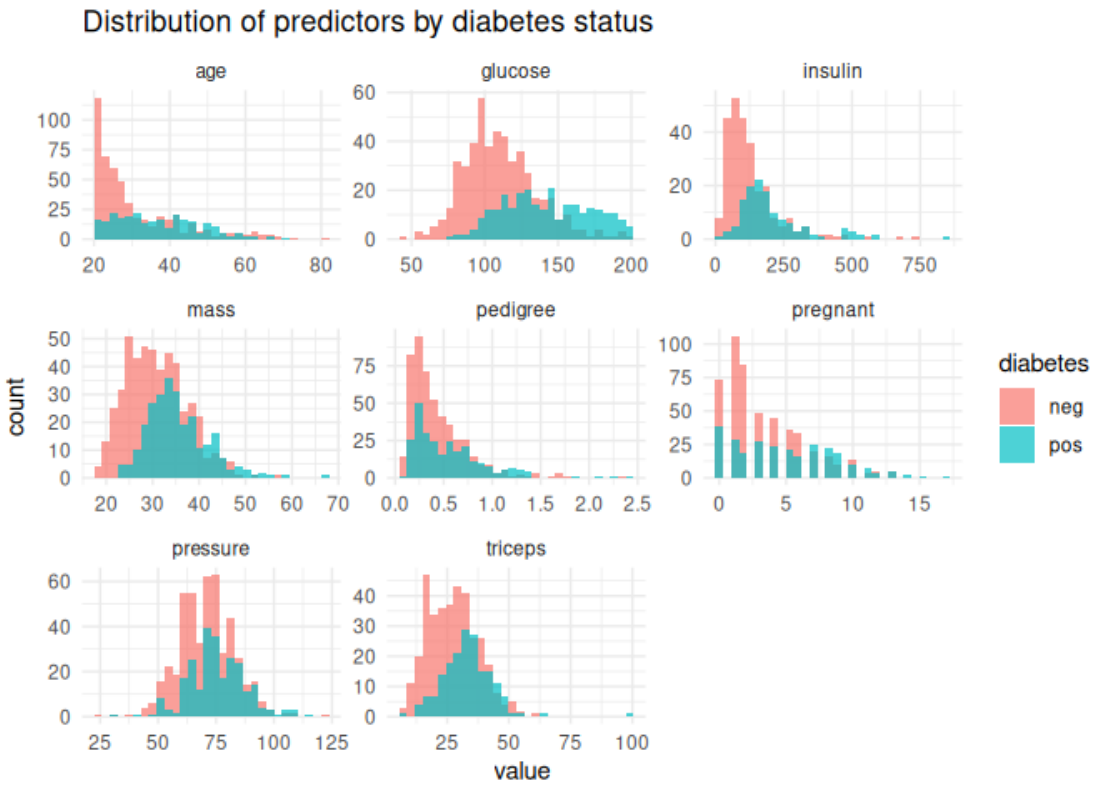
ggplot(df, aes(diabetes, fill = diabetes)) + geom_bar() +
  labs(title = "Class distribution", x = "Diabetes", y = "Count") +
  theme_minimal() + theme(legend.position = "none")
```



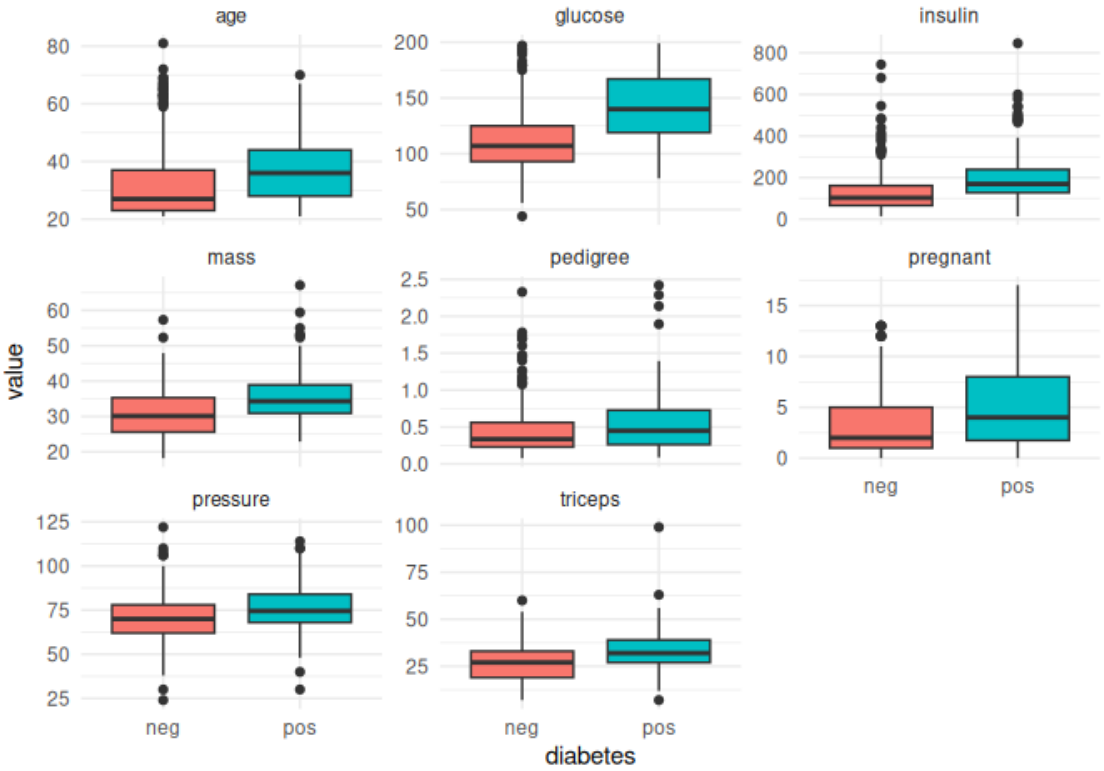
3.3 Distributions

```
long <- tidyr::pivot_longer(df, cols = -diabetes,
                            names_to = "variable", values_to = "value")

ggplot(long, aes(value, fill = diabetes)) +
  geom_histogram(bins = 30, alpha = 0.7, position = "identity", na.rm = TRUE)
 +
  facet_wrap(~variable, scales = "free") +
  labs(title = "Distribution of predictors by diabetes status") +
  theme_minimal()
```




```r
ggplot(long, aes(diabetes, value, fill = diabetes)) +
  geom_boxplot(na.rm = TRUE) +
  facet_wrap(~variable, scales = "free_y") +
  theme_minimal() + theme(legend.position = "none")
```



3.4 Normality checks

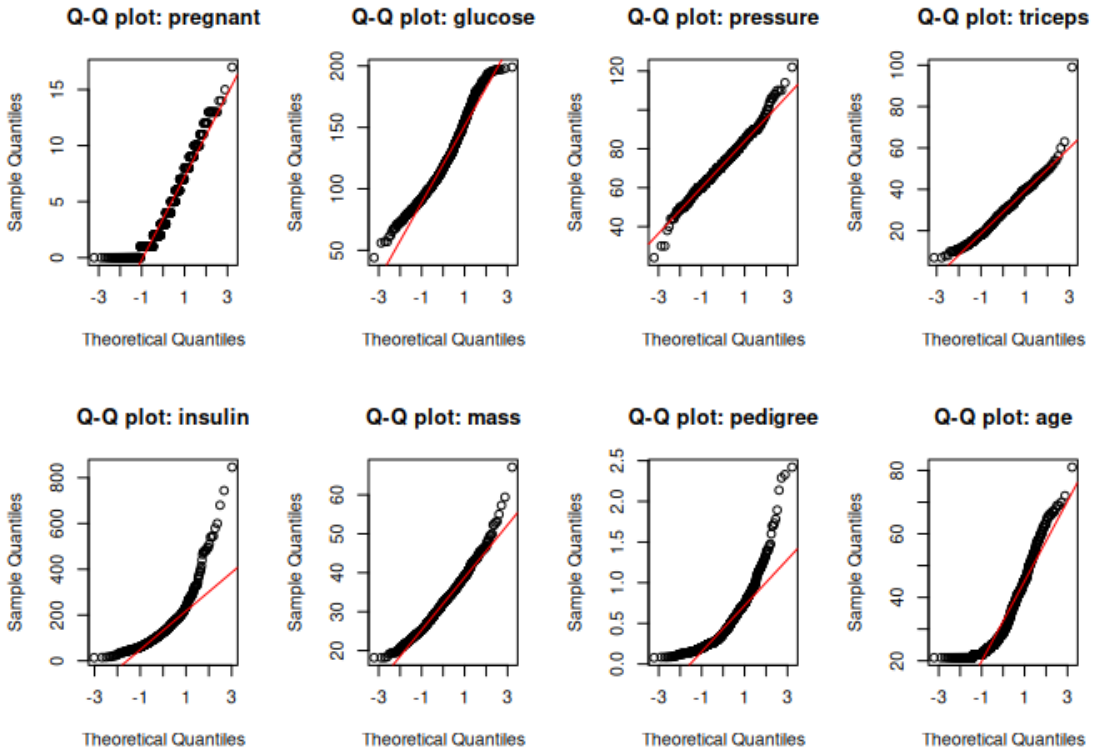
H0: the variable is normally distributed. H1: it is not.

```
num_vars <- setdiff(names(df), "diabetes")

normality <- sapply(num_vars, function(v) shapiro.test(na.omit(df[[v]]))$p.va
lue)
round(normality, 6)

## pregnant  glucose pressure  triceps  insulin     mass pedigree      age
##  0.0e+00  0.0e+00  9.5e-05  0.0e+00  0.0e+00  0.0e+00  0.0e+00  0.0e+00

par(mfrow = c(2, 4))
for (v in num_vars) {
  qqnorm(df[[v]], main = paste("Q-Q plot:", v))
  qqline(df[[v]], col = "red")
}
```




```
par(mfrow = c(1, 1))
```

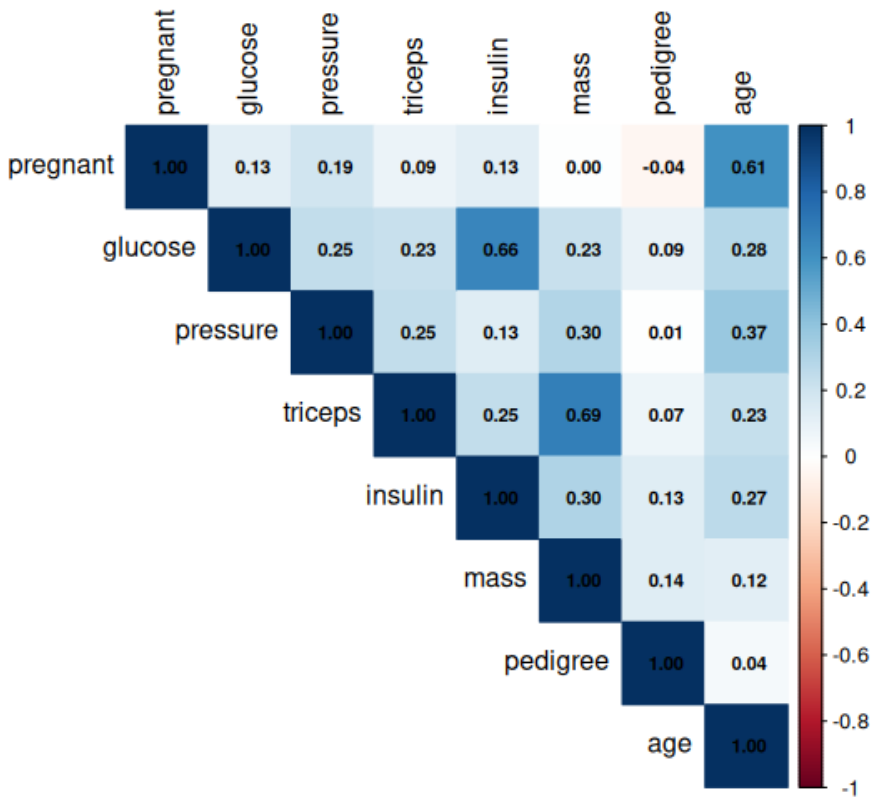
Note: with n = 768 the Shapiro-Wilk test is very sensitive, so Q-Q plots should be read together with the p-values. Most variables are right-skewed, which supports using non-parametric tests alongside t-tests.

3.5 Correlation analysis

```
num_df  <- df[, num_vars]
cor_mat <- cor(num_df, use = "pairwise.complete.obs", method = "spearman")
round(cor_mat, 2)
```

```
##          pregnant glucose pressure triceps insulin mass pedigree  age
## pregnant     1.00    0.13     0.19    0.09    0.13 0.00    -0.04 0.61
## glucose      0.13    1.00     0.25    0.23    0.66 0.23     0.09 0.28
## pressure     0.19    0.25     1.00    0.25    0.13 0.30     0.01 0.37
## triceps      0.09    0.23     0.25    1.00    0.25 0.69     0.07 0.23
## insulin      0.13    0.66     0.13    0.25    1.00 0.30     0.13 0.27
## mass         0.00    0.23     0.30    0.69    0.30 1.00     0.14 0.12
## pedigree    -0.04    0.09     0.01    0.07    0.13 0.14     1.00 0.04
## age          0.61    0.28     0.37    0.23    0.27 0.12     0.04 1.00
```

```
corrplot(cor_mat, method = "color", type = "upper", addCoef.col = "black",
         number.cex = 0.7, tl.col = "black")
```




```
cor.test(df$glucose, df$insulin, method = "spearman", exact = FALSE)
```

```
##
##  Spearman's rank correlation rho
##
## data:  df$glucose and df$insulin
## S = 3451562, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##       rho
## 0.6588133
```

```
cor.test(df$age, df$pregnant, method = "spearman", exact = FALSE)
```

```
##
##  Spearman's rank correlation rho
##
## data:  df$age and df$pregnant
## S = 29654123, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##       rho
## 0.6072163
```

3.6 Hypothesis tests

Test 1: Is mean glucose different between diabetic and non-diabetic groups?

- H0: mean glucose (pos) = mean glucose (neg)
- H1: mean glucose (pos) is different from mean glucose (neg)

```
t.test(glucose ~ diabetes, data = df)

##
##  Welch Two Sample t-test
##
## data:  glucose by diabetes
## t = -14.884, df = 466.02, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group neg and gro
up pos is not equal to 0
## 95 percent confidence interval:
##  -35.85757 -27.49380
## sample estimates:
## mean in group neg mean in group pos
##          110.6439          142.3195

wilcox.test(glucose ~ diabetes, data = df, exact = FALSE)

##
##  Wilcoxon rank sum test with continuity correction
##
## data:  glucose by diabetes
## W = 27394, p-value < 2.2e-16
## alternative hypothesis: true location shift is not equal to 0
```

Test 2: The same comparison for every predictor (Wilcoxon rank-sum)

```
wil <- sapply(num_vars, function(v) {
  wilcox.test(df[[v]] ~ df$diabetes, exact = FALSE)$p.value
})
wil_tbl <- data.frame(variable = num_vars,
                      p_value = signif(wil, 3),
                      p_adj_bonferroni = signif(p.adjust(wil, "bonferroni"),
3))
wil_tbl[order(wil_tbl$p_value), ]

##          variable  p_value p_adj_bonferroni
## glucose   glucose 1.31e-40         1.04e-39
## age           age 1.14e-17         9.14e-17
## mass         mass 1.82e-17         1.46e-16
## insulin   insulin 7.48e-14         5.98e-13
## triceps   triceps 6.96e-10         5.57e-09
## pregnant pregnant 3.75e-08         3.00e-07
## pedigree pedigree 1.20e-06         9.57e-06
## pressure pressure 1.63e-06         1.30e-05
```

Test 3: Is diabetes associated with age group? (Chi-square)

- H0: diabetes status is independent of age group
- H1: diabetes status depends on age group

```r
age_group <- cut(df$age, breaks = c(20, 30, 40, 50, Inf),
                 labels = c("21-30", "31-40", "41-50", "51+"))
tab <- table(age_group, df$diabetes)
tab
```

```
##
## age_group neg pos
##     21-30 327  90
##     31-40  81  76
##     41-50  49  64
##     51+    43  38
```

```r
chisq.test(tab)
```

```
##
##  Pearson's Chi-squared test
##
## data:  tab
## X-squared = 73.809, df = 3, p-value = 6.522e-16
```

```r
round(prop.table(tab, 1), 3)
```

```
##
## age_group   neg   pos
##     21-30 0.784 0.216
##     31-40 0.516 0.484
##     41-50 0.434 0.566
##     51+   0.531 0.469
```

Test 4: Variance check (Levene / Brown-Forsythe style test) for glucose

```r
gl <- df[!is.na(df$glucose), ]
gl$abs_dev <- abs(gl$glucose - ave(gl$glucose, gl$diabetes, FUN = median))
anova(lm(abs_dev ~ diabetes, data = gl))
```

```
## Analysis of Variance Table
##
## Response: abs_dev
##            Df Sum Sq Mean Sq F value    Pr(>F)
## diabetes    1   5896  5896.1  23.212 1.752e-06 ***
## Residuals 761 193307   254.0
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4. Model building

4.1 Train / test split

Missing values are imputed **inside** the cross-validation folds (median imputation via `preProcess`), which avoids data leakage.

```r
set.seed(123)
idx   <- createDataPartition(df$diabetes, p = 0.8, list = FALSE)
train <- df[idx, ]
test  <- df[-idx, ]
dim(train); dim(test)
```

```
## [1] 615   9
```

```
## [1] 153   9
```

```r
round(prop.table(table(train$diabetes)), 3)
```

```
##
##  neg  pos
## 0.65 0.35
```

```r
round(prop.table(table(test$diabetes)), 3)
```

```
##
##   neg   pos
## 0.654 0.346
```

4.2 Cross-validation setup

```r
ctrl <- trainControl(method = "cv", number = 10,
                     classProbs = TRUE,
                     summaryFunction = twoClassSummary,
                     savePredictions = "final")
# NOTE: for twoClassSummary the first factor level ("neg") is treated as the
# event, so Sens/Spec in the CV table refer to "neg". ROC (AUC) is unaffected.
```

4.3 Model 1: Logistic regression

```r
set.seed(123)
fit_glm <- train(diabetes ~ ., data = train,
                 method = "glm", family = "binomial",
                 metric = "ROC", trControl = ctrl,
                 preProcess = c("medianImpute", "center", "scale"),
                 na.action = na.pass)
fit_glm
```

```
## Generalized Linear Model
##
## 615 samples
##   8 predictor
##   2 classes: 'neg', 'pos'
##
## Pre-processing: median imputation (8), centered (8), scaled (8)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 553, 553, 553, 554, 554, 554, ...
## Resampling results:
##
```

```
##   ROC        Sens   Spec
##   0.8201569  0.875  0.5534632
```

```
summary(fit_glm$finalModel)
```

```
##
## Call:
## NULL
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.844330   0.107597  -7.847 4.26e-15 ***
## pregnant     0.351205   0.122110   2.876  0.00403 **
## glucose      1.142803   0.131644   8.681  < 2e-16 ***
## pressure    -0.131260   0.116154  -1.130  0.25845
## triceps      0.005898   0.153431   0.038  0.96934
## insulin     -0.091151   0.146012  -0.624  0.53245
## mass         0.596744   0.136116   4.384 1.16e-05 ***
## pedigree     0.266280   0.105474   2.525  0.01158 *
## age          0.149549   0.124342   1.203  0.22909
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 796.05  on 614  degrees of freedom
## Residual deviance: 588.12  on 606  degrees of freedom
## AIC: 606.12
##
## Number of Fisher Scoring iterations: 5
```

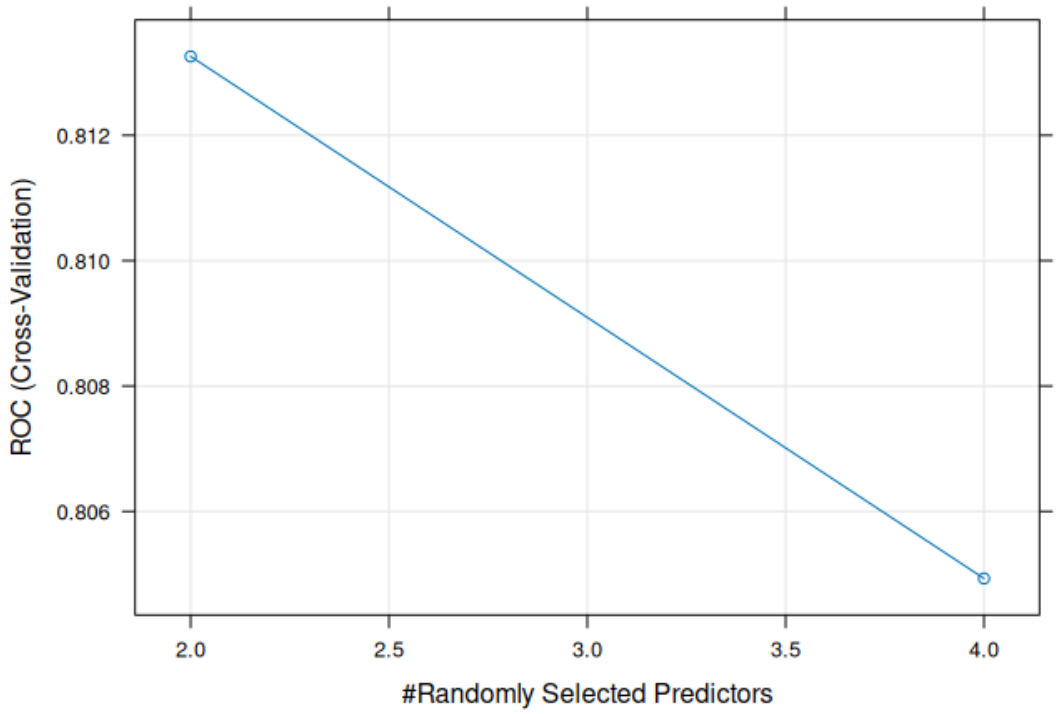
4.4 Model 2: Random forest (comparison model)

```
set.seed(123)
fit_rf <- train(diabetes ~ ., data = train,
                method = "rf", metric = "ROC", trControl = ctrl,
                tuneGrid = expand.grid(mtry = c(2, 4)),
                ntree = 200,
                preProcess = c("medianImpute"),
                na.action = na.pass)
fit_rf
```

```
## Random Forest
##
## 615 samples
##   8 predictor
##   2 classes: 'neg', 'pos'
##
## Pre-processing: median imputation (8)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 553, 553, 553, 554, 554, 554, ...
```

```
## Resampling results across tuning parameters:
##
##   mtry  ROC        Sens    Spec
##   2     0.8132576  0.8425  0.5575758
##   4     0.8049297  0.8400  0.5666667
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 2.
```

```
plot(fit_rf)
```



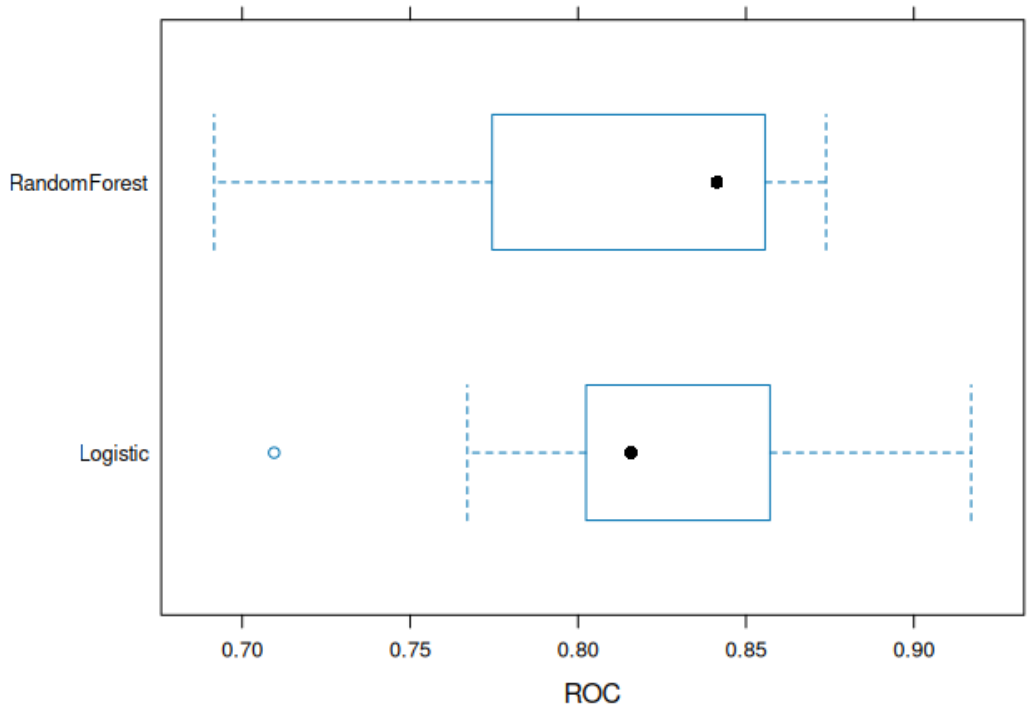
4.5 Cross-validated comparison

```
res <- resamples(list(Logistic = fit_glm, RandomForest = fit_rf))
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: Logistic, RandomForest
## Number of resamples: 10
##
## ROC
##                  Min.   1st Qu.    Median      Mean   3rd Qu.      Max. N
A's
## Logistic     0.7095238 0.8032062 0.8156926 0.8201569 0.8553571 0.9170455
```

```
  0
## RandomForest 0.6916667 0.7772727 0.8413420 0.8132576 0.8553030 0.8738636
  0
##
## Sens
##                Min. 1st Qu. Median   Mean 3rd Qu.  Max. NA's
## Logistic     0.825   0.850 0.8750 0.8750 0.90000 0.925    0
## RandomForest 0.775   0.825 0.8375 0.8425 0.86875 0.925    0
##
## Spec
##                   Min.   1st Qu.    Median      Mean  3rd Qu.      Max. NA
's
## Logistic     0.3636364 0.4599567 0.5811688 0.5534632 0.612013 0.7727273
 0
## RandomForest 0.2857143 0.5059524 0.5811688 0.5575758 0.612013 0.7727273
 0

bwplot(res, metric = "ROC")
```



5. Evaluation on the held-out test set

```
pred_glm <- predict(fit_glm, newdata = test, na.action = na.pass)
prob_glm <- predict(fit_glm, newdata = test, type = "prob", na.action = na.pa
ss)[, "pos"]
```

```r
cm_glm   <- confusionMatrix(pred_glm, test$diabetes, positive = "pos")
cm_glm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction neg pos
##        neg  91  21
##        pos   9  32
##
##                Accuracy : 0.8039
##                  95% CI : (0.7321, 0.8636)
##     No Information Rate : 0.6536
##     P-Value [Acc > NIR] : 3.3e-05
##
##                   Kappa : 0.5426
##
##  Mcnemar's Test P-Value : 0.04461
##
##             Sensitivity : 0.6038
##             Specificity : 0.9100
##          Pos Pred Value : 0.7805
##          Neg Pred Value : 0.8125
##              Prevalence : 0.3464
##          Detection Rate : 0.2092
##    Detection Prevalence : 0.2680
##       Balanced Accuracy : 0.7569
##
##        'Positive' Class : pos
##
```

```r
pred_rf <- predict(fit_rf, newdata = test, na.action = na.pass)
prob_rf <- predict(fit_rf, newdata = test, type = "prob", na.action = na.pass)
[, "pos"]
cm_rf   <- confusionMatrix(pred_rf, test$diabetes, positive = "pos")
cm_rf
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction neg pos
##        neg  89  18
##        pos  11  35
##
##                Accuracy : 0.8105
##                  95% CI : (0.7393, 0.8692)
##     No Information Rate : 0.6536
##     P-Value [Acc > NIR] : 1.46e-05
##
##                   Kappa : 0.568
```

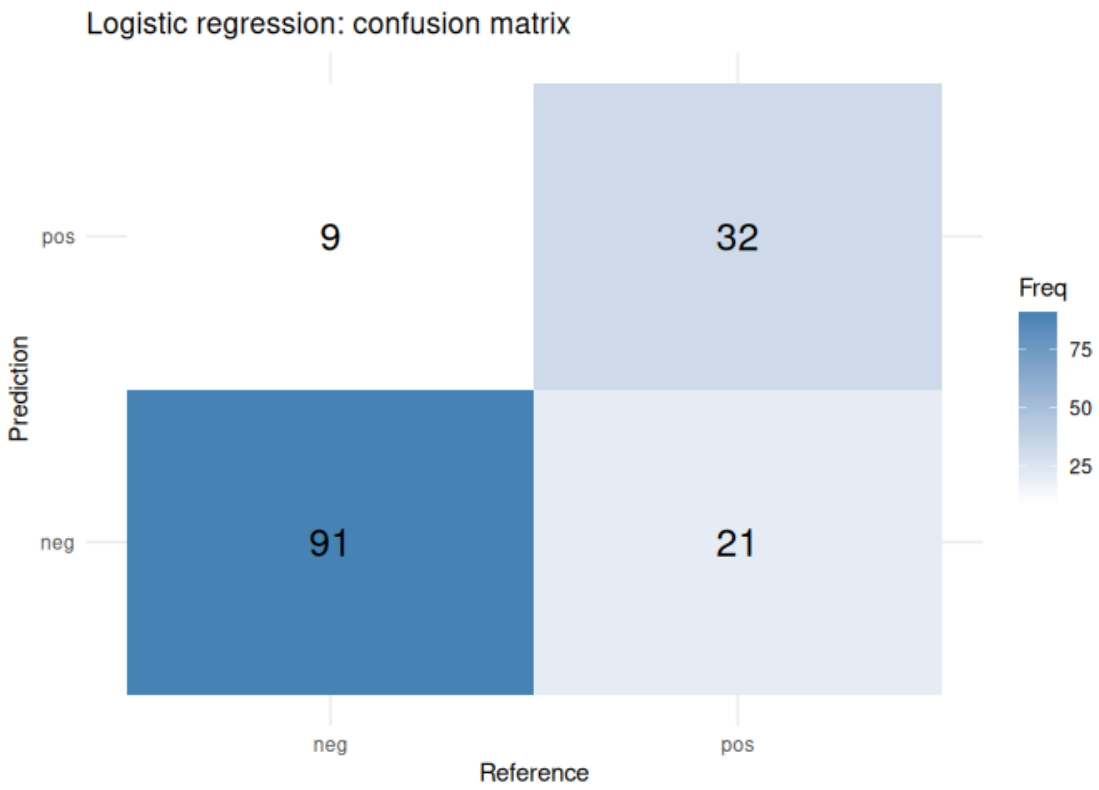
```
##
##  Mcnemar's Test P-Value : 0.2652
##
##             Sensitivity : 0.6604
##             Specificity : 0.8900
##          Pos Pred Value : 0.7609
##          Neg Pred Value : 0.8318
##              Prevalence : 0.3464
##          Detection Rate : 0.2288
##    Detection Prevalence : 0.3007
##       Balanced Accuracy : 0.7752
##
##        'Positive' Class : pos
##
```

```
metrics <- data.frame(
  Model       = c("Logistic", "RandomForest"),
  Accuracy    = c(cm_glm$overall["Accuracy"],     cm_rf$overall["Accuracy"]),
  Sensitivity = c(cm_glm$byClass["Sensitivity"],  cm_rf$byClass["Sensitivity
"]),
  Specificity = c(cm_glm$byClass["Specificity"],  cm_rf$byClass["Specificity
"]),
  Precision   = c(cm_glm$byClass["Precision"],    cm_rf$byClass["Precision"]),
  F1          = c(cm_glm$byClass["F1"],           cm_rf$byClass["F1"])
)
metrics[, -1] <- round(metrics[, -1], 3)
metrics
```

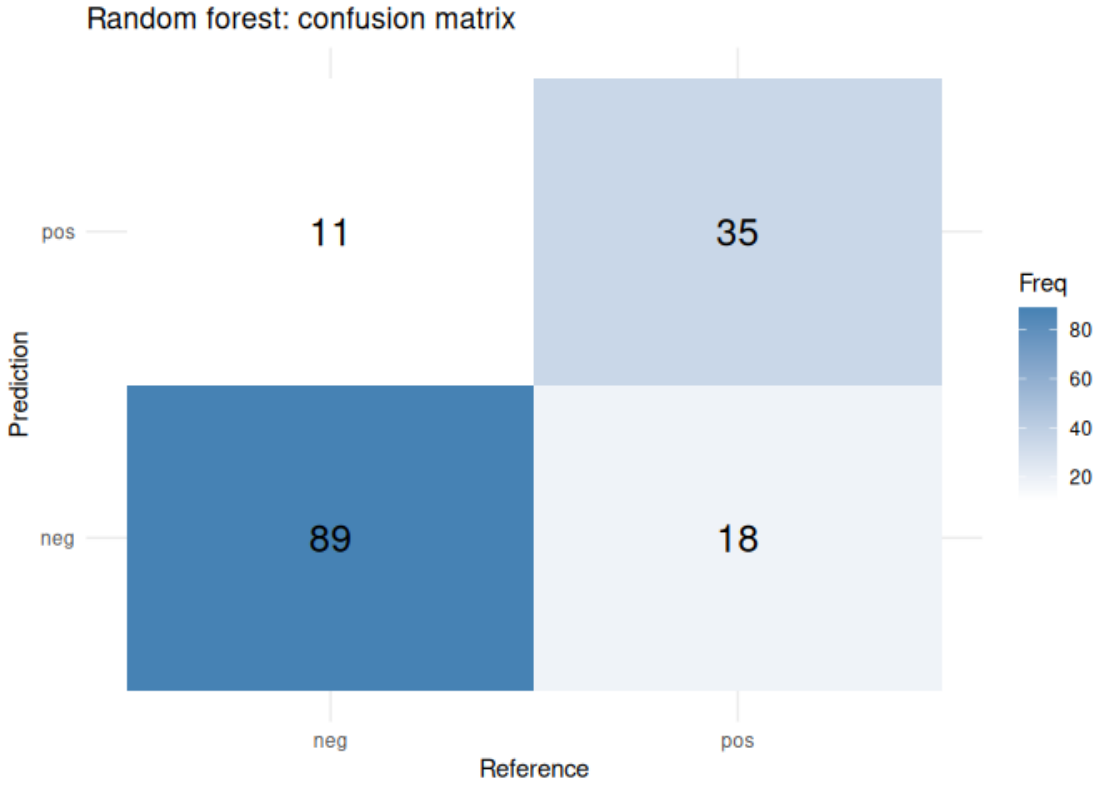
```
##          Model Accuracy Sensitivity Specificity Precision    F1
## 1     Logistic    0.804       0.604        0.91     0.780 0.681
## 2 RandomForest    0.810       0.660        0.89     0.761 0.707
```

5.1 Confusion matrix plot

```
plot_cm <- function(cm, title) {
  ggplot(as.data.frame(cm$table), aes(Reference, Prediction, fill = Freq)) +
    geom_tile() + geom_text(aes(label = Freq), size = 6) +
    scale_fill_gradient(low = "white", high = "steelblue") +
    labs(title = title) + theme_minimal()
}
plot_cm(cm_glm, "Logistic regression: confusion matrix")
```




```
plot_cm(cm_rf,  "Random forest: confusion matrix")
```



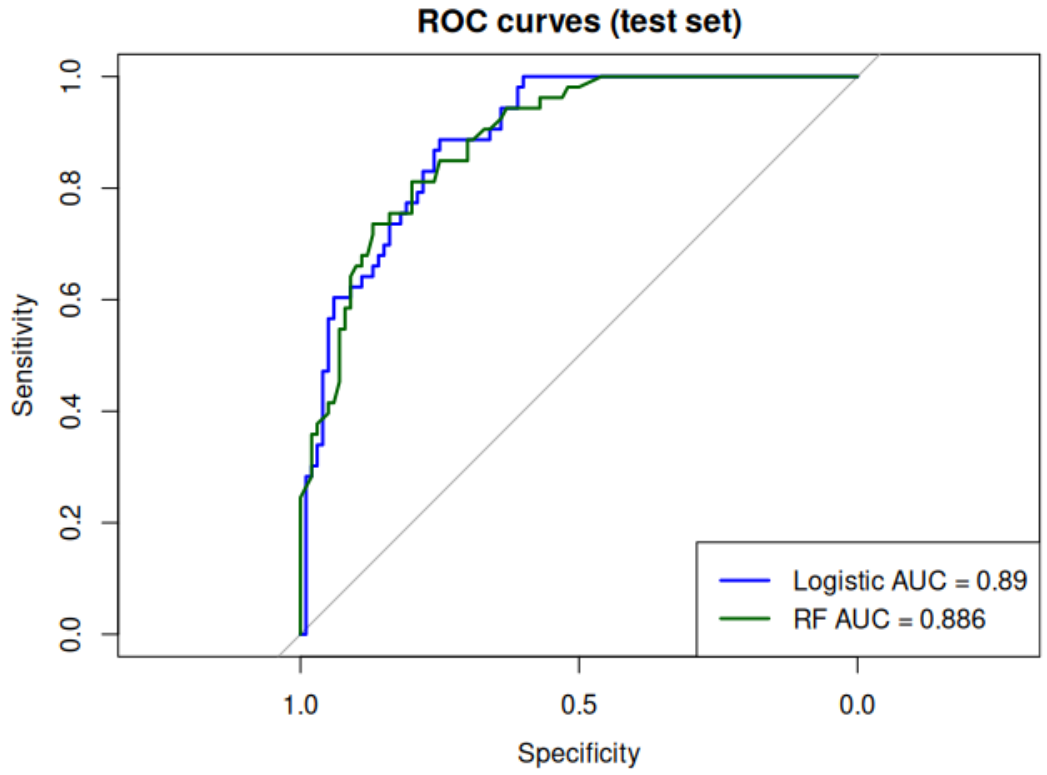
5.2 ROC curves and AUC

```
roc_glm <- roc(test$diabetes, prob_glm, levels = c("neg", "pos"), direction =
 "<")
roc_rf  <- roc(test$diabetes, prob_rf,  levels = c("neg", "pos"), direction =
 "<")
auc(roc_glm); auc(roc_rf)

## Area under the curve: 0.8904

## Area under the curve: 0.8863

plot(roc_glm, col = "blue", lwd = 2, main = "ROC curves (test set)")
plot(roc_rf, col = "darkgreen", lwd = 2, add = TRUE)
legend("bottomright",
       legend = c(paste("Logistic AUC =", round(auc(roc_glm), 3)),
                  paste("RF AUC =", round(auc(roc_rf), 3))),
       col = c("blue", "darkgreen"), lwd = 2)
```



6. Diagnostic analysis

6.1 Logistic regression: odds ratios

Predictors were standardized, so each odds ratio is per 1 standard deviation increase.

```r
glm_final <- fit_glm$finalModel
ci <- exp(confint.default(glm_final))
or_tbl <- data.frame(
  term       = names(coef(glm_final)),
  odds_ratio = round(exp(coef(glm_final)), 3),
  CI_low     = round(ci[, 1], 3),
  CI_high    = round(ci[, 2], 3),
  p_value    = round(summary(glm_final)$coefficients[, 4], 4)
)
rownames(or_tbl) <- NULL
or_tbl
```

```
##          term odds_ratio CI_low CI_high p_value
## 1 (Intercept)      0.430  0.348   0.531  0.0000
## 2    pregnant      1.421  1.118   1.805  0.0040
## 3     glucose      3.136  2.422   4.059  0.0000
## 4    pressure      0.877  0.698   1.101  0.2585
## 5     triceps      1.006  0.745   1.359  0.9693
## 6     insulin      0.913  0.686   1.215  0.5325
## 7        mass      1.816  1.391   2.372  0.0000
## 8    pedigree      1.305  1.061   1.605  0.0116
## 9         age      1.161  0.910   1.482  0.2291
```

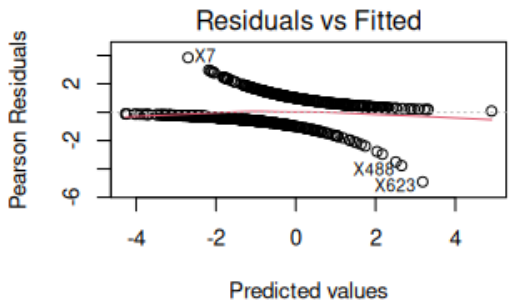
6.2 Multicollinearity (VIF, computed manually)

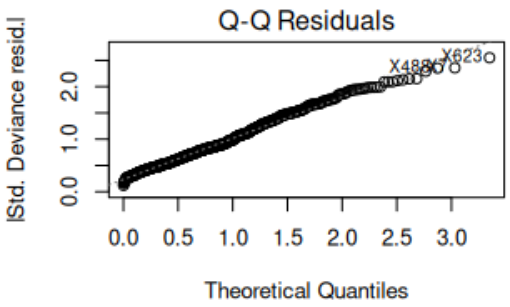
```r
X <- model.matrix(glm_final)[, -1]
vif_vals <- sapply(seq_len(ncol(X)), function(i) {
  r2 <- summary(lm(X[, i] ~ X[, -i]))$r.squared
  1 / (1 - r2)
})
names(vif_vals) <- colnames(X)
round(vif_vals, 2)    # values above 5 suggest a problem
```

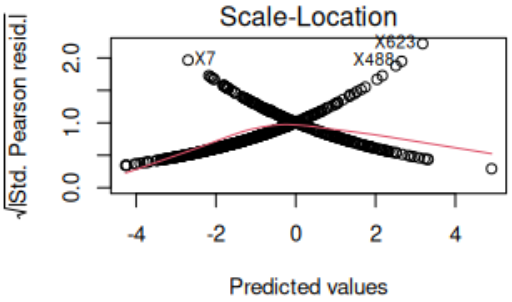
```
## pregnant  glucose pressure  triceps  insulin     mass pedigree      age
##     1.46     1.35     1.23     1.48     1.23     1.62     1.05     1.66
```

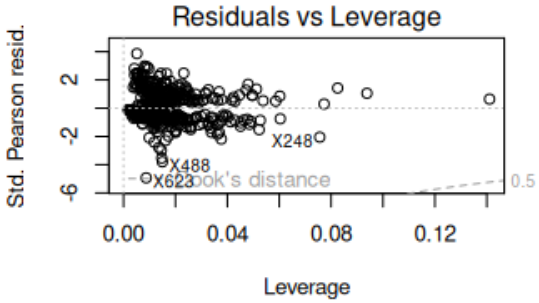
6.3 Residual diagnostics

```r
par(mfrow = c(2, 2))
plot(glm_final)
```



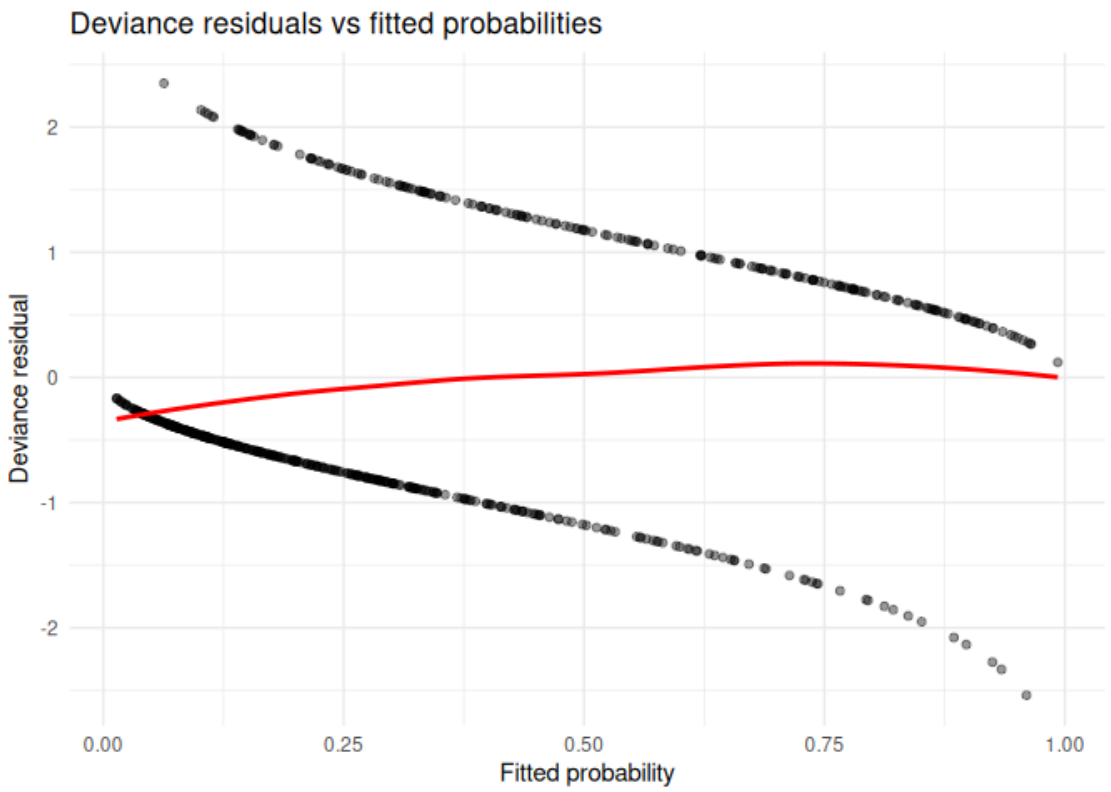




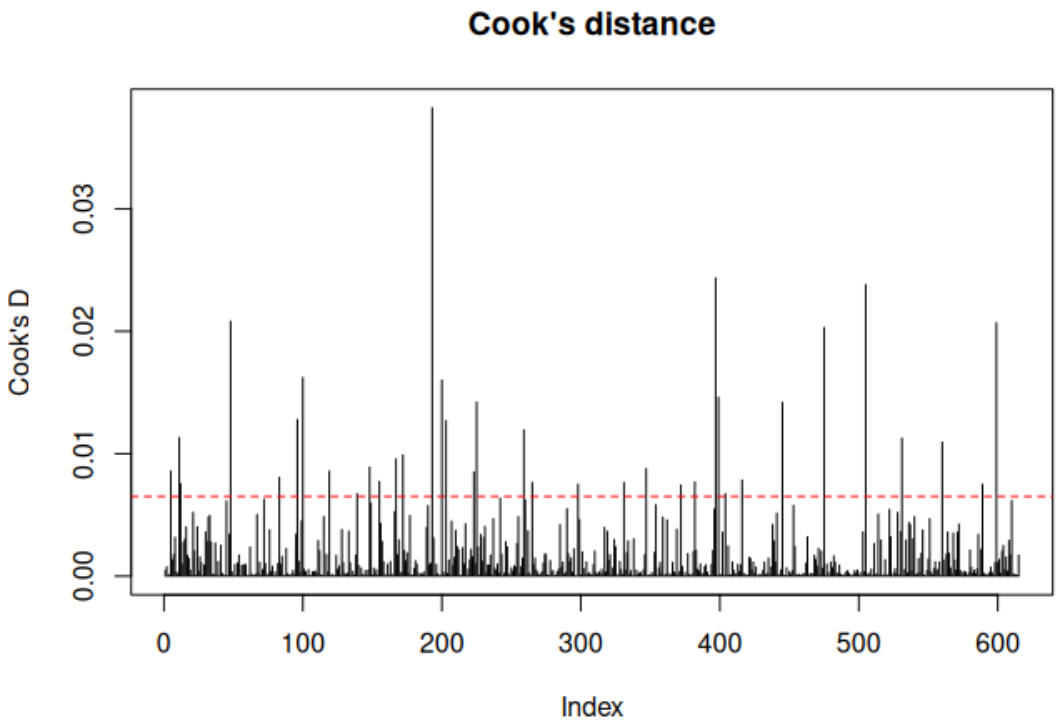



```r
par(mfrow = c(1, 1))

res_df <- data.frame(fitted  = fitted(glm_final),
                     dev_res = residuals(glm_final, type = "deviance"))
ggplot(res_df, aes(fitted, dev_res)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "loess", formula = y ~ x, se = FALSE, colour = "red")
+
  labs(title = "Deviance residuals vs fitted probabilities",
       x = "Fitted probability", y = "Deviance residual") +
  theme_minimal()
```




```r
cd <- cooks.distance(glm_final)
plot(cd, type = "h", main = "Cook's distance", ylab = "Cook's D")
abline(h = 4 / length(cd), col = "red", lty = 2)
```

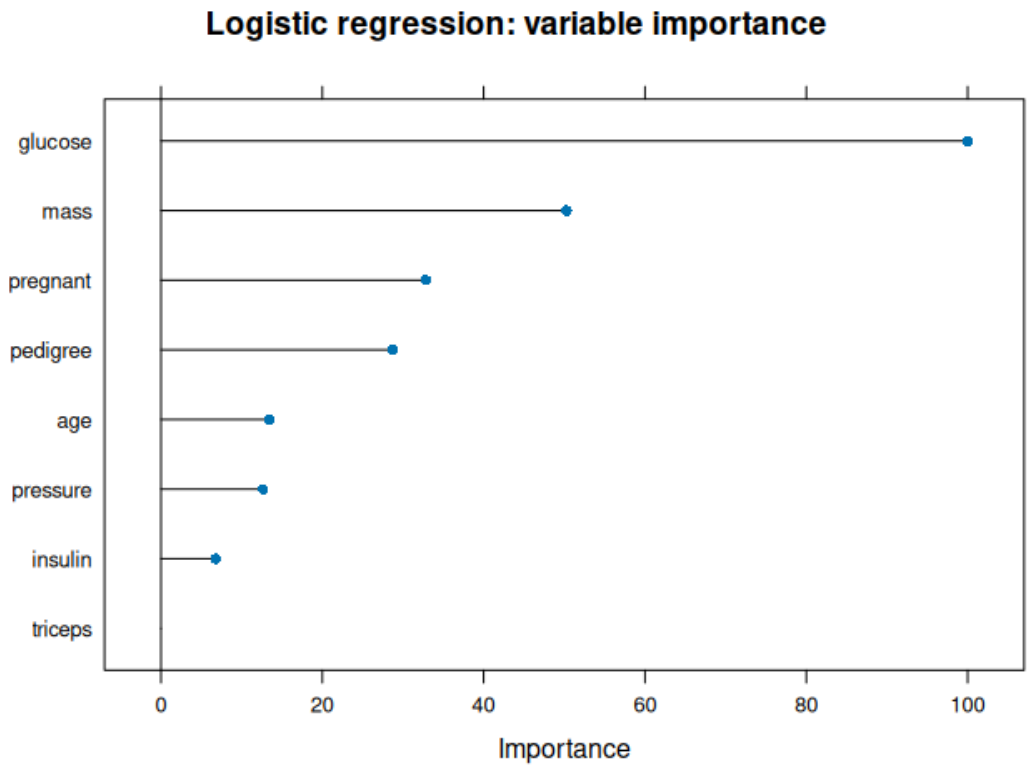



```
sum(cd > 4 / length(cd))    # number of potentially influential points
```

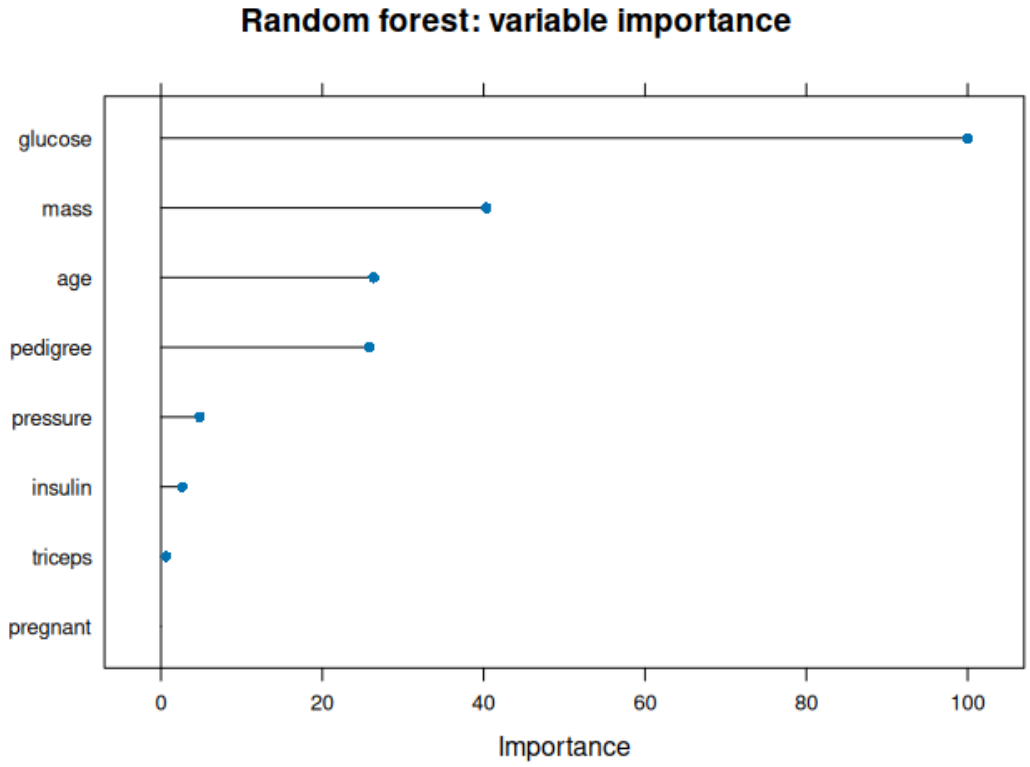
```
## [1] 36
```

6.4 Variable importance

```
plot(varImp(fit_glm), main = "Logistic regression: variable importance")
```




```r
plot(varImp(fit_rf),  main = "Random forest: variable importance")
```



7. Optimization and improvement

7.1 Threshold tuning (improving sensitivity)

```
best <- coords(roc_glm, "best", best.method = "youden",
               ret = c("threshold", "sensitivity", "specificity"))
best
```

```
##   threshold sensitivity specificity
## 1 0.2702626   0.8867925        0.75
```

```
thr <- as.numeric(best[["threshold"]][1])
new_pred <- factor(ifelse(prob_glm >= thr, "pos", "neg"), levels = c("neg", "
pos"))
confusionMatrix(new_pred, test$diabetes, positive = "pos")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction neg pos
##        neg  75   6
##        pos  25  47
##
##                Accuracy : 0.7974
##                  95% CI : (0.7249, 0.858)
##     No Information Rate : 0.6536
##     P-Value [Acc > NIR] : 7.169e-05
##
##                   Kappa : 0.5873
##
##  Mcnemar's Test P-Value : 0.001225
##
##             Sensitivity : 0.8868
##             Specificity : 0.7500
##          Pos Pred Value : 0.6528
##          Neg Pred Value : 0.9259
##              Prevalence : 0.3464
##          Detection Rate : 0.3072
##    Detection Prevalence : 0.4706
##       Balanced Accuracy : 0.8184
##
##        'Positive' Class : pos
##
```

7.2 Other ideas to try

- Better imputation (kNN or `mice`) instead of median imputation
- Feature engineering (e.g. BMI categories, glucose x age interaction)
- Class balancing (SMOTE or down-sampling) and tuning with a recall-focused metric

- Other models: regularized logistic regression (glmnet), gradient boosting (xgboost), SVM
- Repeated cross-validation for more stable estimates

8. Interpretation of Results

Hypothesis tests:-

Mean glucose was higher in diabetic patients (142.3 vs 110.6; Welch t = -14.88, p < 2.2e-16). A variance test was significant (p = 1.75e-06), so the unequal-variance Welch test was appropriate. All eight predictors differed significantly between groups after Bonferroni correction, and diabetes prevalence was strongly related to age group (chi-square = 73.8, p = 6.5e-16), rising from 21.6% in the 21-30 group to 56.6% in the 41-50 group.

Correlations:-

The strongest relationships were mass-triceps (0.69), glucose-insulin (0.66) and age-pregnant (0.61). VIF values were all below 1.7, so multicollinearity is not a concern.

Model performance:-

Logistic regression and random forest performed very similarly (cross-validated AUC 0.820 vs 0.813; test AUC 0.890 vs 0.886; accuracy 0.804 vs 0.810). Both beat the 65.4% majority-class baseline. Because the test set has only 153 patients, the small differences between the models are not meaningful. Note that Sens/Spec in the cross-validation tables refer to the “neg” class, while the test-set confusion matrices use “pos” as the positive class.

Strengths and weaknesses:-

Specificity was high (0.91 and 0.89) but sensitivity was low (0.60 and 0.66), meaning about one third to 40% of diabetic patients were missed at the default 0.5 threshold. Lowering the threshold to 0.27 raised sensitivity to 0.887 (specificity 0.75, accuracy 0.797), which is preferable for screening.

Key predictors:-

In the logistic model, glucose had the largest effect (odds ratio 3.14 per standard deviation, 95% CI 2.42-4.06), followed by mass (1.82), pregnant (1.42) and pedigree (1.31). Pressure, triceps, insulin and age were not significant once the other variables were included.

9. Key Insights & Key Features

Insights

- **Group differences:** After Bonferroni correction, these predictors differed significantly between diabetic and non-diabetic patients: pregnant, glucose, pressure, triceps, insulin, mass, pedigree, age.

- **Glucose:** The Welch t-test for glucose gave p = <2e-16, so mean glucose differs between the two groups.
- **Age and diabetes:** The chi-square test of age group vs diabetes gave p = 6.52e-16.
- **Model performance:** On the test set, logistic regression reached an accuracy of 0.804 and AUC of 0.89; random forest reached an accuracy of 0.81 and AUC of 0.886.
- **Baseline comparison:** Always predicting the majority class would give an accuracy of 0.654, so the models should be judged against this number.
- **Sensitivity vs specificity:** Logistic regression detected 60.4% of diabetic cases (specificity 91%); random forest detected 66% (specificity 89%).
- **Best model by AUC:** logistic regression.

Features

The three most important predictors in the logistic regression model were **glucose**, **mass** and **pregnant** (see the variable importance plot in section 6.4). The table below summarizes all features used.

Feature	Description	Type
pregnant	Number of times pregnant	Numeric predictor
glucose	Plasma glucose concentration (2-hour oral glucose tolerance test)	Numeric predictor
pressure	Diastolic blood pressure (mm Hg)	Numeric predictor
triceps	Triceps skin fold thickness (mm)	Numeric predictor
insulin	2-hour serum insulin (mu U/ml)	Numeric predictor
mass	Body mass index	Numeric predictor
pedigree	Diabetes pedigree function	Numeric predictor
age	Age in years	Numeric predictor
diabetes	Target: pos / neg	Binary target

10. Conclusion & Limitations

This project analysed the Pima Indians Diabetes dataset using hypothesis tests, correlation and normality analysis, and two classification models evaluated with 10-fold cross-validation and a held-out test set. The two models performed almost identically, with test AUC of 0.89 for logistic regression and 0.886 for random forest.

The hypothesis tests showed that all predictors differ between groups individually, while the multivariable model identified glucose, BMI, number of pregnancies and the pedigree function as independent predictors, with glucose by far the strongest. Both models performed comparably (test AUC about 0.89), so the simpler and more interpretable logistic regression is the preferable choice.

Limitations

The main limitations are the small sample size, median imputation of missing values, and moderate class imbalance. Future work could use multiple imputation, feature engineering, class balancing, threshold tuning (a threshold near 0.27 raised sensitivity from 0.60 to 0.89) and other algorithms such as gradient boosting.

11.Screenshots of the RStudio Workflow

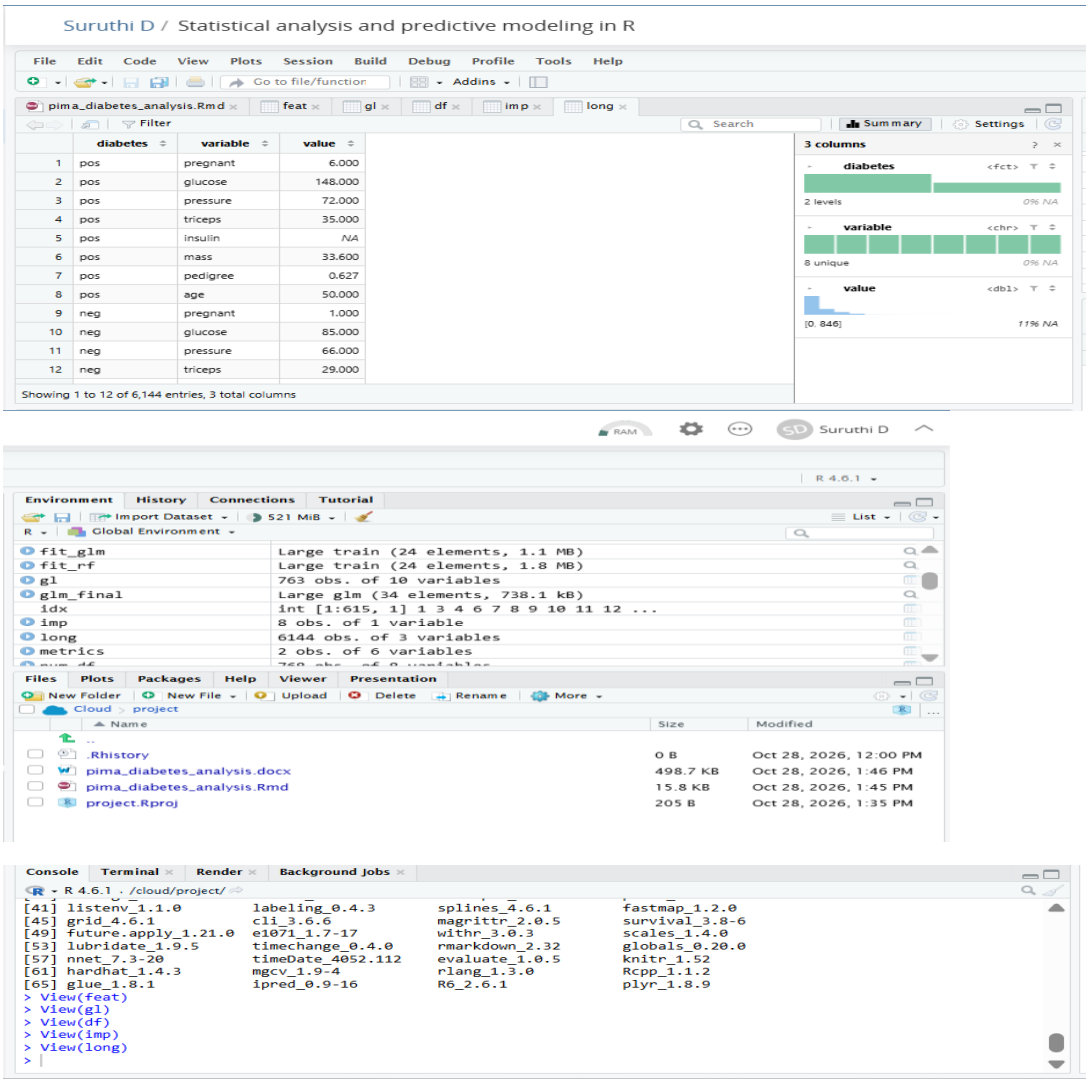


Fig: *Project running in RStudio on Posit Cloud, showing the Rmd file, data objects and knitted output.*

Appendix

A. Data dictionary and source

Dataset: `PimaIndiansDiabetes2` (R package `mlbench`), 768 observations, 8 predictors and 1 binary target. Original source: National Institute of Diabetes and Digestive and Kidney Diseases.

B. Missing values per variable

```
data.frame(Missing = colSums(is.na(df)),
           Percent = round(colMeans(is.na(df)) * 100, 1))
```

```
##          Missing Percent
## pregnant       0     0.0
## glucose        5     0.7
## pressure      35     4.6
## triceps      227    29.6
## insulin      374    48.7
## mass          11     1.4
## pedigree       0     0.0
## age            0     0.0
## diabetes       0     0.0
```

C. Full odds ratio table

```
knitr::kable(or_tbl)
```

term	odds_ratio	CI_low	CI_high	p_value
(Intercept)	0.430	0.348	0.531	0.0000
pregnant	1.421	1.118	1.805	0.0040
glucose	3.136	2.422	4.059	0.0000
pressure	0.877	0.698	1.101	0.2585
triceps	1.006	0.745	1.359	0.9693
insulin	0.913	0.686	1.215	0.5325
mass	1.816	1.391	2.372	0.0000
pedigree	1.305	1.061	1.605	0.0116
age	1.161	0.910	1.482	0.2291

D. Test-set performance table

```
knitr::kable(metrics)
```

Model	Accuracy	Sensitivity	Specificity	Precision	F1
Logistic	0.804	0.604	0.91	0.780	0.681
RandomForest	0.810	0.660	0.89	0.761	0.707

E. Session information

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
##  [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
##  [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
## [10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C
##
## time zone: UTC
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] randomForest_4.7-1.2 corrplot_0.95        pROC_1.19.1
## [4] caret_7.0-1          lattice_0.22-9       tidyr_1.3.2
## [7] dplyr_1.2.1          ggplot2_4.0.3        mlbench_2.1-11
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.6         xfun_0.62            recipes_1.4.0
##  [4] vctrs_0.7.3          tools_4.6.1          generics_0.1.4
##  [7] stats4_4.6.1         parallel_4.6.1       proxy_0.4-29
## [10] tibble_3.3.1         pkgconfig_2.0.3      ModelMetrics_1.2.2.2
## [13] Matrix_1.7-5         data.table_1.18.6.1  RColorBrewer_1.1-3
## [16] S7_0.2.2             lifecycle_1.0.5      compiler_4.6.1
## [19] farver_2.1.2         stringr_1.6.0        codetools_0.2-20
## [22] htmltools_0.5.9      class_7.3-23         yaml_2.3.12
## [25] prodlim_2026.03.11   pillar_1.11.1        MASS_7.3-65
## [28] gower_1.0.2          iterators_1.0.14     rpart_4.1.27
## [31] foreach_1.5.2        nlme_3.1-169         parallelly_1.48.0
## [34] lava_1.10            tidyselect_1.2.1     digest_0.6.39
## [37] stringi_1.8.9        future_1.80.0        reshape2_1.4.5
## [40] purrr_1.2.2          listenv_1.1.0        labeling_0.4.3
## [43] splines_4.6.1        fastmap_1.2.0        grid_4.6.1
## [46] cli_3.6.6            magrittr_2.0.5       survival_3.8-6
## [49] e1071_1.7-17         future.apply_1.21.0  withr_3.0.3
## [52] scales_1.4.0         lubridate_1.9.5      timechange_0.4.0
## [55] rmarkdown_2.32       globals_0.20.0       nnet_7.3-20
```

```
## [58] timeDate_4052.112    evaluate_1.0.5       knitr_1.52
## [61] hardhat_1.4.3        mgcv_1.9-4           rlang_1.3.0
## [64] Rcpp_1.1.2           glue_1.8.1           ipred_0.9-16
## [67] R6_2.6.1             plyr_1.8.9
```